Bases de Datos 2

Teoría 01

Julián Grigera

Presentación

Docentes

Profesores

- Julián Grigera
- Federico Orlando
- Alejandra Lliteras

Jefes de Trabajos Prácticos

- Natalia Correa
- Federico Di Claudio

Información General

Contactos

ACCESIBILIDAD	A.Paola Amadeo	accesibilidad@info.unlp.edu.ar pamadeo@info.unlp.edu.ar
ORIENTACIÓN AL ESTUDIANTE	Ana Ungaro	orientación.estudiantil@info.unlp.edu.ar anaungaro@info.unlp.edu.ar
GÉNERO	Sofía Martin	ddhhygenero@info.unlp.edu.ar smartin@info.unlp.edu.ar
ASESORAMIENTO PEDAGÓGICO	Anahí Almán	direccion.pedagogica@info.unlp.edu.ar

Presentación

Horarios, Aprobación etc

Moodle
https://catedras.linti.unlp.edu.ar/
Registrarse a Bases de Datos 2

Modalidad
- Examen parcial con 2 recuperatorios
- Prácticas con ejercicios teórico/prácticos (máquina)

Consultas
Inicio semana 23/03

- Martes 08:00 - 9:30 Aula 8
- Martes 19:00 - 20:30 Sala de PC
- Miércoles 19:00 - 20:30 Aula 4
- Viernes 08:00 - 9:30 Aula 10B

No son obligatorias
Se puede asistir a cualquier horario.

Promoción

Condiciones

- El parcial no se publica con nota, pero al aprobar en 1ra o 2da fecha, se revisa el mismo para establecer nota de final.
- Aprobando el parcial en **1ra fecha**
  - máxima nota posible **10**
- Aprobando el parcial en **2da fecha**
  - máxima nota posible **9**

Bases de Datos 2

Teoría 01

Julián Grigera

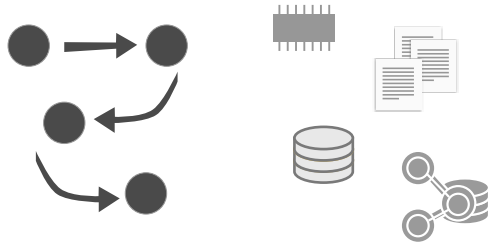
Hasta ahora usan BDs aisladas, generalmente relacionales
Vamos a hablar de su aplicabilidad
Ergo, vamos a ver qué **opciones tenemos** y **no** tenemos
Desarrollar criterios para decidir

Bases de Datos
RDMBS

Bases de Datos relacionales:
- Es un paradigma muy conocido y utilizado - hay mucha experiencia en el mercado
- SQL es un estándar que permite migrar fácilmente
- Hay muchos sistemas “legacy”
- No parece que se vaya a reemplazar en el mediano plazo

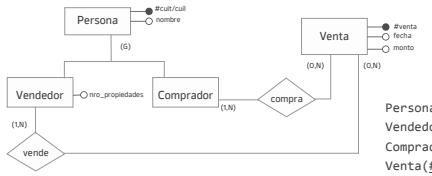
Orientación a Objetos:
- Muy presente en la industria

Persistencia
Por qué / dónde guardo los objetos?



- En un sistema OO ¿Dónde puedo guardar objetos?
    - Memoria: sirve, pero no tiene durabilidad ni permite distribución fácilmente
    - Archivos: poco soporte (ej. no hay índices, transacciones)
    - BBDD(R): modelo fundamentalmente diferente del paradigma OO
    - BDOO: pocas opciones, poca experiencia/recursos

Modelo Relacional
Cuántas relaciones hay?




```
Persona(cuit, nombre)
Vendedor(cuit, nro_propiedades)
Comprador(cuit)
Venta(#venta, fecha, monto)
Vende(cuil, #venta)
Compra(cuil, #venta)
```

Pensando en el modelo relacional ¿Cuántas relaciones hay en este diagrama?
- 2
- 6
- 5
- **ninguna**

La trampa: es un diagrama E/R, el modelo relacional es otra cosa

El modelo relacional es muy poderoso, pero también muy simple respecto de OO

Diferencia de Impedancia

Entre paradigma OO y modelo relacional

Diferencia de OO respecto de RDBMS

- OO no contempla normalmente manejo de tx
- Se representan grafos de objetos sin límite aparente

Diferencias de RDBMS respecto de OO

- Cuando hay múltiples relaciones y recursión no es eficiente
- Recuperar datos dispersados incurre en varios JOINs
- No soporta Jerarquías - se traduce en más JOINs

Existen entonces los ORMs (Object-Relational Mapper) que buscan resolver esta diferencia

---

Diferencia de Impedancia

Mapping

Se podría pensar que estamos volviendo al paradigma procedural -> separando comportamiento de los datos

No exactamente:

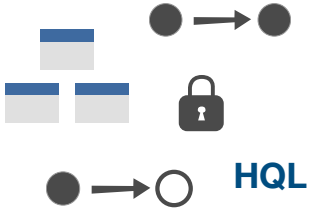
- Seguimos programando OO
- Buscamos **olvidarnos** de la BD subyacente

Existen principios para reforzar esto.

Antes, veamos cómo es un ORM real

Hibernate

Hibernate
Introducción



Hibernate es el ORM de referencia de JPA
Soporte para Jerarquías, Polimorfismo, Relaciones, Tx, Lazy, HQL

Hibernate
Mapeo XML

```
package prueba;
public class Persona {
      private Long idPersona;
      private String nombre;
      private Date fechaNacimiento;
      public Date getFechaNacimiento() {
         return fechaNacimiento;
      }
      public void
         setFechaNacimiento(Date
         fechaNacimiento) {
         this.fechaNacimiento =
         fechaNacimiento;
      }
      public Long getIdPersona() {
         return idPersona;
      }
      public void setIdPersona(Long
         idPersona) {
         this.idPersona = idPersona;
      }
      public String getNombre() {
         return nombre;
      }
      public void setNombre(String
         nombre) {
         this.nombre = nombre;
      }
}
```

Persona.hbm.xml (Archivo Mapping)

```
<hibernate-mapping package="prueba">
  <class name="Persona" table="PERSONA">

    <id name="idPersona" column="ID_PERSONA">
                <generator class="native"/>
    </id>

    <property name="nombre" not-null="true"/>

    <property name="fechaNacimiento"/>

  </class>
</hibernate-mapping>
```

Pueden definirse mapeos mediante XML
- Es más tedioso de definir y mantener que su alternativa (annotations)
+ Es mejor para la independencia (las clases del modelo quedan menos “contaminadas” por código de persistencia)

Hibernate
Mapeo Annotations

```
@Entity(name = "Contact")
public static class Contact {

        @Id
        private Integer id;

        private Name name;

        private String notes;

        private URL website;

        private boolean starred;

        //Getters and setters are omitted for brevity
}
```

Annotations permite un código más simple pero rompe el principio de independencia
Se aprovecha el concepto de “Convención sobre Configuración”

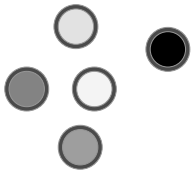
Hibernate
POJOs

Reglas a seguir (POJO -> Plain Old Java Object)

- Implementar un **constructor sin argumentos**. *
- Proveer un **identificador**
- Clases **no-finales**
- **Setters y getters** para los campos persistentes

Principios para ORMs

ORM
Principios



Ortogonalidad

Independencia

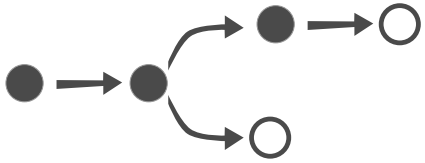
Principio de Ortogonalidad: todo objeto debe poder ser persistido más allá de su tipo (más allá de los *transient* objects que es una elección)
Principio de Independencia: el sistema OO debe saber lo menos posible sobre la tecnología de persistencia subyacente

ORM
Principio de Independencia

Si vemos un código típico (aunque antiguo) de Hibernate encontramos:
- H/SQL embebido
- Annotations
- Tx Explícitas
- “Saves”

Todo esto va contra el principio de independencia

ORM
Persistencia por Alcance



Con esto, se reduce el nro de saves explícitos
Ideal para soportar el principio independencia
¿Complicaciones posibles?
¿Implementación?